

Design and Analysis of Algorithm – II UNIT

2 marks:

1. What is Brute force approach of problem solving? Give an example.

The brute force approach is a problem-solving strategy that involves trying every possible solution until the correct one is found. It is a straightforward and intuitive method, but it can be inefficient for large problems due to its exponential time complexity.

Example: Finding the shortest path between two cities in a road network.

To find the largest number in a list of numbers.

2. List any four importance of Brute force.

Simplicity: The brute force approach is easy to understand and implement.

Guaranteed solution: The brute force approach always finds a solution if one exists.

Benchmarking: The brute force algorithm can be used as a benchmark to compare the performance of other algorithms.

Educational value: The brute force approach can help students understand the concept of algorithm efficiency.

3. Write the number of operations and time complexity of selection sort.

• Number of operations = $\frac{(n-1)n}{2}$

• Time Complexity : $O(n^2)$

4. Write the number of operations and time complexity of bubble sort.

• Number of operations = $\frac{(n-1)n}{2}$

• Time Complexity : $O(n^2)$

5. What do you mean by Sequential Search? Write its time complexity.

Sequential search, also known as linear search, is a simple search algorithm that searches for a target element in a list by examining each element in the list sequentially until the target element is found or the end of the list is reached.

The time complexity of sequential search is $O(n)$, where n is the number of elements in the list.

6. Write the worst case and average case complexity of Brute force String Matching.

• Worst case complexity: $O(mn)$

• Average case complexity: $O(n+m) = O(n)$

7. Write the number of operations and time complexity of Closest-Pair Problem.

• Number of operations : $(n-1)n$

• Time Complexity : $O(n^2)$

8. Define Convex and Convex hull.

Convex : A set of points in space is convex if for any two points in the set, the line segment connecting them lies entirely within the set.

Convex hull : The convex hull of a set of points is the smallest convex set that contains all the points in the set.

9. List any two example algorithms of the brute force approach.

- Sequential search
- Brute force string matching

10. What do you mean by convex hull problem? Write its time complexity.

The convex-hull problem is the problem of constructing the convex hull for a given set S of n points. To solve it, we need to find the points that will serve as the vertices of the polygon in question.

Time Complexity : $O(n^3)$

11. Define Exhaustive search mechanism. Give example.

Exhaustive search is simply a brute-force approach to combinatorial problems. It suggests generating each and every element of the problem's domain, selecting those of them that satisfy all the constraints, and then finding a desired element.

Example : Traveling Salesman Problem (TSP), Knapsack problem

12. What do you mean by Knapsack Problem? Write its time complexity.

The problem is to find the subset of a set of items that maximizes the total value of the items subject to a weight constraint.

Time Complexity : $O(N*W)$ where 'N' - number of weight elements, 'W' - capacity of the knapsack

13. What do you mean by Travelling Salesman Problem? Write its time complexity.

The problem asks to find the shortest tour through a given set of n cities that visits each city exactly once before returning to the city where it started. The problem can be conveniently modeled by a weighted graph, with the graph's vertices representing the cities and the edge weights specifying the distances.

Time Complexity : $O(n!)$ where n is the number of cities.

14. What do you mean by Job assignment problem? Write its time complexity.

The job assignment problem is a classic optimization problem in computer science. The problem is to assign a set of n jobs to n machines in such a way that the total completion time is minimized.

Time Complexity : $O(n!)$

Long Answer Questions

1. Write an algorithm to sort N numbers using Selection sort. Derive the number of operations and time complexity.

```
ALGORITHM SelectionSort( $A[0..n - 1]$ )
    //Sorts a given array by selection sort
    //Input: An array  $A[0..n - 1]$  of orderable elements
    //Output: Array  $A[0..n - 1]$  sorted in ascending order
    for  $i \leftarrow 0$  to  $n - 2$  do
         $min \leftarrow i$ 
        for  $j \leftarrow i + 1$  to  $n - 1$  do
            if  $A[j] < A[min]$   $min \leftarrow j$ 
        swap  $A[i]$  and  $A[min]$ 
```

The input's size is given by the number of elements n .

The algorithm's basic operation is the key comparison $A[j] < A[\min]$.

The number of times it is executed depends only on the array's size and is given by the following sum:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}$$

Thus, selection sort is a $O(n^2)$ algorithm on all inputs. The number of key swaps is $O(n)$.

2. Write an algorithm to sort N numbers by applying Bubble sort. Derive the number operations and time complexity.

ALGORITHM *BubbleSort*($A[0..n-1]$)
 //Sorts a given array by bubble sort
 //Input: An array $A[0..n-1]$ of orderable elements
 //Output: Array $A[0..n-1]$ sorted in ascending order
for $i \leftarrow 0$ **to** $n-2$ **do**
 for $j \leftarrow 0$ **to** $n-2-i$ **do**
 if $A[j+1] < A[j]$ **swap** $A[j]$ and $A[j+1]$

The input's size is given by the number of elements n .

The algorithm's basic operation is the key comparison.

The number of times it is executed depends only on the array's size and is given by the following sum:

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1] \\ &= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2). \end{aligned}$$

3. Write and describe the Sequential search algorithm.

Sequential search, also known as linear search, is a simple method to find a specific value in a list. It works by starting at the beginning of the list and comparing each element with the value we're searching for, until the desired element is found or the end of the list is reached.

ALGORITHM *SequentialSearch2*($A[0..n], K$)
 //Implements sequential search with a search key as a sentinel
 //Input: An array A of n elements and a search key K
 //Output: The index of the first element in $A[0..n-1]$ whose value is
 // equal to K or -1 if no such element is found
 $A[n] \leftarrow K$
 $i \leftarrow 0$
while $A[i] \neq K$ **do**
 $i \leftarrow i + 1$
if $i < n$ **return** i
else return -1

The time complexity of sequential search is $O(n)$, where n is the number of elements in the list. This is because in the worst-case scenario, we have to check every element in the list. Therefore, while sequential search is easy to implement, it can be inefficient for large lists.

4. Write and describe Brute force String Matching Algorithm.

Brute force string matching is a simple string matching algorithm that searches for a pattern string in a text string by comparing the pattern string to each substring of the text string of the same length.

ALGORITHM *BruteForceStringMatch*($T[0..n-1]$, $P[0..m-1]$)

```

//Implements brute-force string matching
//Input: An array  $T[0..n-1]$  of  $n$  characters representing a text and
//       an array  $P[0..m-1]$  of  $m$  characters representing a pattern
//Output: The index of the first character in the text that starts a
//        matching substring or  $-1$  if the search is unsuccessful

    for  $i \leftarrow 0$  to  $n - m$  do
         $j \leftarrow 0$ 
        while  $j < m$  and  $P[j] = T[i + j]$  do
             $j \leftarrow j + 1$ 
        if  $j = m$  return  $i$ 
    return  $-1$ 

```

The brute force string matching algorithm is a simple and intuitive algorithm, but it is also inefficient for large problems. This is because the time complexity of the algorithm is $O(mn)$, which means that the algorithm can take a long time to run for long text strings and pattern strings.

5. Write and explain the algorithm for Closest-Pair Problem. Derive its complexity.

The closest-pair problem is to find the pair of points in a set of points that are closest to each other.

ALGORITHM *BruteForceClosestPoints*(P)

```

//Finds two closest points in the plane by brute force
//Input: A list  $P$  of  $n$  ( $n \geq 2$ ) points  $P_1 = (x_1, y_1), \dots, P_n = (x_n, y_n)$ 
//Output: Indices  $index1$  and  $index2$  of the closest pair of points
 $dmin \leftarrow \infty$ 
for  $i \leftarrow 1$  to  $n - 1$  do
    for  $j \leftarrow i + 1$  to  $n$  do
         $d \leftarrow \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2)$  //sqrt is the square root function
        if  $d < dmin$ 
             $dmin \leftarrow d$ ;  $index1 \leftarrow i$ ;  $index2 \leftarrow j$ 
return  $index1, index2$ 

```

The basic operation of the algorithm is computing the Euclidean distance between two points. The number of times it will be executed can be computed as follows:

$$\begin{aligned}
 C(n) &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2 = 2 \sum_{i=1}^{n-1} (n-i) \\
 &= 2[(n-1) + (n-2) + \dots + 1] = (n-1)n \in \Theta(n^2)
 \end{aligned}$$

6. Write and explain the algorithm for Convex Hull problem.

The Convex Hull is the smallest convex polygon that contains all the points of a given set in the plane. The problem is to compute the Convex Hull of a set of points.

```
BruteForceConvexHull(points):
    n = number of points
    convex_hull = empty list

    for each subset of 3 points (p1, p2, p3):
        if points form a left turn:
            Add p1, p2, and p3 to convex_hull

    return convex_hull
```

The time complexity of this brute force algorithm is $O(n^3)$, where N is the number of points. This is because the algorithm iterates through all possible subsets of three points.

7. Explain Travelling Salesman Problem by exhaustive search with an example.

The problem asks to find the shortest tour through a given set of n cities that visits each city exactly once before returning to the city where it started. The problem can be conveniently modeled by a weighted graph, with the graph's vertices representing the cities and the edge weights specifying the distances.

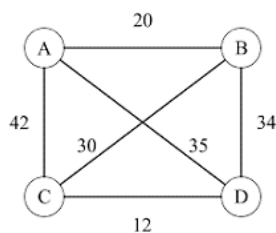
Example: Refer Q.No. 9

8. Write a note on Knapsack Problem with an example.

Given n items of known weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$ and a knapsack of capacity W , find the most valuable subset of the items that fit into the knapsack.

Example: Refer Q.No. 12

9. Find the optimal solution for the Traveling Salesman problem using exhaustive search method by considering 'A' as the starting city.

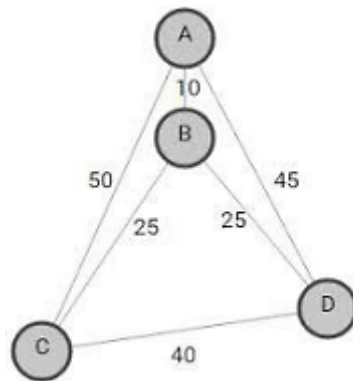


Tour	Length
a -> b -> c -> d -> a	$l=20+30+12+35=97$ optimal
a -> b -> d -> c -> a	$l=20+34+12+42=98$
a -> c -> b -> d -> a	$l=42+30+34+35=141$
a -> c -> d -> b -> a	$l=42+12+34+20=108$
a -> d -> b -> c -> a	$l=35+34+30+42=141$
a -> d -> c -> b -> a	$l=35+12+30+20=97$ optimal

So its optimal solution is :

a -> b -> c -> d -> a	}	97
a -> d -> c -> b -> a		

10. Find the optimal solution for the Traveling Salesman problem using exhaustive search method by considering 'C' as the starting city.

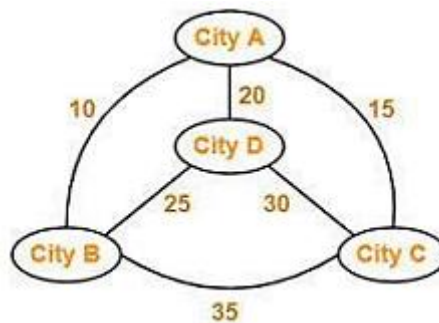


Possible routes:

- C → A → B → D → C (total distance = 125)
- C → A → D → B → C (total distance = 145)
- C → B → A → D → C (total distance = 120)
- C → B → D → A → C (total distance = 145)
- C → D → A → B → C (total distance = 120)
- C → D → B → A → C (total distance = 125)

So its optimal solution is : C → B → A → D → C (total distance = 120)
 C → D → A → B → C (total distance = 120)

11. Find the optimal solution for the Traveling Salesman problem using exhaustive search method by considering 'City D' as the starting city.



Possible routes:

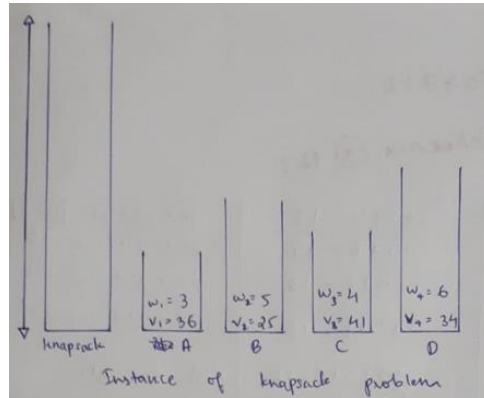
- D → A → B → C → D (total distance = 95)
- D → A → C → B → D (total distance = 95)
- D → B → A → C → D (total distance = 80)
- D → B → C → A → D (total distance = 95)
- D → C → A → B → D (total distance = 80)
- D → C → B → A → D (total distance = 95)

So its optimal solution is : D → B → A → C → D (total distance = 80)
 D → C → A → B → D (total distance = 80)

12. Consider the Knapsack problem with the following inputs. Solve the problem using exhaustive search. Enumerate all possibilities and indicate unfeasible solutions and Optimal solution. Knapsack total capacity $W=15\text{kg}$

Items	A	B	C	D
Weight(kg)	3	5	4	6
Value	36	25	41	34

Ans:



Subset	Weight (kg)	Value
A	3	36
B	5	25
C	4	41
D	6	34
A + B	8	61
A + C	7	77
A + D	9	70
B + C	9	66
B + D	11	59
C + D	10	75
A + B + C	12	102
A + B + D	14	95
A + C + D	13	111
B + C + D	15	100
A + B + C + D	18	136

The optimal solution is the subset with the highest total value that does not exceed the knapsack capacity. In this case, the optimal solution is A + C + D, with a total weight of 13kg and a total value of 111.

13. Consider the Knapsack problem with the following inputs. Solve the problem using exhaustive search. Enumerate all possibilities and indicate unfeasible solutions and optimal solution. Knapsack total capacity $W=20\text{kg}$

Items	Item1	Item2	Item3	Item4
Weight(kg)	8	10	7	4
Value	40	45	65	30

Items Included	Total Weight (kg)	Total Value	Feasible?
Item1	8	40	Yes
Item2	10	45	Yes
Item3	7	65	Yes
Item4	4	30	Yes
Item1, Item2	18	85	Yes
Item1, Item3	15	105	Yes
Item1, Item4	12	70	Yes
Item2, Item3	17	110	Yes
Item2, Item4	14	75	Yes
Item3, Item4	11	95	Yes
Item1, Item2, Item3	25	150	No
Item1, Item2, Item4	22	115	No
Item1, Item3, Item4	19	135	Yes
Item2, Item3, Item4	21	140	No
Item1, Item2, Item3, Item4	28	175	No

From the above combinations, the combination with the highest total value that doesn't exceed the total capacity of the knapsack is {Item1, Item3, Item4} with a total weight of 19 kg and a total value of 135. Therefore, the optimal solution to this problem is to take items Item1, Item3, and Item4.

14. Consider the Job Assignment problem with the following inputs. Solve the problem using exhaustive search. Calculate Minimal total cost to complete to all jobs one job by each person.

	JOB1	JOB2	JOB3	JOB4
Person-1	9	2	7	8
Person-2	6	4	3	7
Person-3	5	8	1	8
Person-4	7	6	9	4

Assignment	Person 1	Person 2	Person 3	Person 4	Total Cost
1	JOB1	JOB2	JOB3	JOB4	18
2	JOB1	JOB2	JOB4	JOB3	30
3	JOB1	JOB3	JOB2	JOB4	24
4	JOB1	JOB3	JOB4	JOB2	26
5	JOB1	JOB4	JOB2	JOB3	33
6	JOB1	JOB4	JOB3	JOB2	23
7	JOB2	JOB1	JOB3	JOB4	13
8	JOB2	JOB1	JOB4	JOB3	25
9	JOB2	JOB3	JOB1	JOB4	14
10	JOB2	JOB3	JOB4	JOB1	20
11	JOB2	JOB4	JOB1	JOB3	23
12	JOB2	JOB4	JOB3	JOB1	17
13	JOB3	JOB1	JOB2	JOB4	25
14	JOB3	JOB1	JOB4	JOB2	27
15	JOB3	JOB2	JOB1	JOB4	20
16	JOB3	JOB2	JOB4	JOB1	26
17	JOB3	JOB4	JOB1	JOB2	25
18	JOB3	JOB4	JOB2	JOB1	29
19	JOB4	JOB1	JOB2	JOB3	31
20	JOB4	JOB1	JOB3	JOB2	21
21	JOB4	JOB2	JOB1	JOB3	26
22	JOB4	JOB2	JOB3	JOB1	20
23	JOB4	JOB3	JOB1	JOB2	22
24	JOB4	JOB3	JOB2	JOB1	26

From the combinations given above, the combination with the lowest total cost is

{Person-1: JOB2, Person-2: JOB1, Person-3: JOB3, Person-4: JOB4} with a total cost of 13.

Therefore, the optimal solution to this problem is to assign

JOB2 to Person-1, JOB1 to Person-2, JOB3 to Person-3, and JOB4 to Person-4.

15. Consider the Job Assignment problem with the following inputs. Solve the problem using exhaustive search. Calculate Minimal total cost to complete to all jobs one job by each person.

	JOB1	JOB2	JOB3
Person-1	9	2	7
Person-2	6	4	3
Person-3	5	8	1

	Person 1	Person 2	Person 3	Total Cost
1	JOB1	JOB2	JOB3	14
2	JOB1	JOB3	JOB2	20
3	JOB2	JOB1	JOB3	9
4	JOB2	JOB3	JOB1	10
5	JOB3	JOB1	JOB2	21
6	JOB3	JOB2	JOB1	16

From the above combinations, the combination with the lowest total cost is

{Person-1: JOB2, Person-2: JOB1, Person-3: JOB3} with a total cost of 9.

Therefore, the optimal solution to this problem is to assign

JOB2 to Person-1, JOB1 to Person-2, and JOB3 to Person-3.